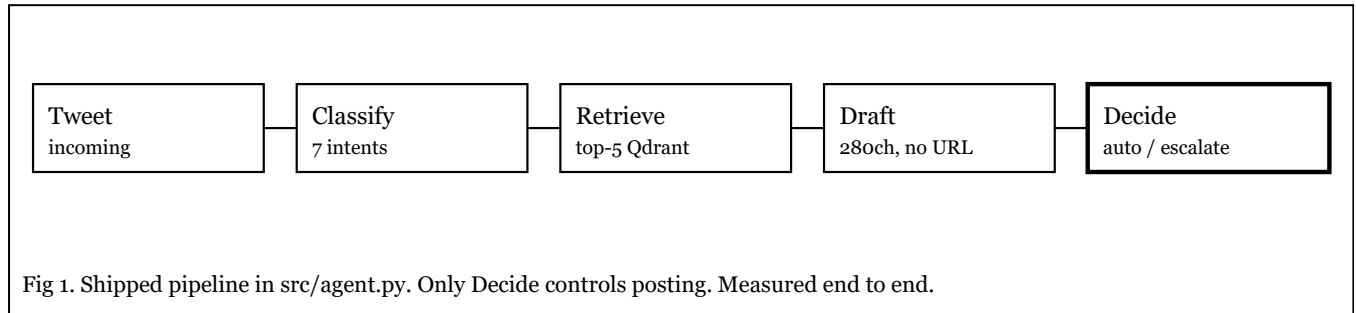


1. Summary

This report presents the Commerce Support AI Agent for enterprise customer support (evaluated on @AmazonHelp on Twitter). The system classifies each customer message into one of seven intents, drafts a reply grounded in how similar cases were historically resolved, and decides whether to auto-handle or escalate to a human with a stated reason. It uses a foundation LLM and Qdrant vector retrieval.



Measured results on 250 human-reviewed threads: intent accuracy 0.820 with 95 percent confidence interval 0.768 to 0.864 and macro-F1 0.809, beating the majority baseline at 0.220 and a fair TF-IDF baseline at 0.508. Escalation recall is 0.981 on golden with one false-auto, but drops to 0.733 on heldout-A and 0.364 on heldout-B. Reply quality is 20.7 out of 25 by blind human scoring versus 19.0 for a template. The LLM judge score of 24.44 is directional only.

All numbers come from results/eval_report.json and docs/escalation_v5.md. Reproduce with run_eval.py --fast --baselines in about three minutes, or full run in about nine minutes with four workers. Offline inspection runs without an API key in under one minute.

2. System

The pipeline has four steps. First, an LLM classifier maps the message to exactly one intent with a confidence score. Second, Qdrant retrieves the top five historical support pairs filtered by that intent using cosine similarity over customer plus reply embeddings. Third, a drafter writes a reply of 280 characters or fewer with a deterministic URL strip, so zero of 250 replies contain links. Fourth, deterministic v5 rules decide escalate or auto-handle, with a narrow LLM tiebreaker only for ultra-short messages of four tokens or fewer.

LLM: classify (JSON) -> raw conf -> `calibrate_confidence()` stored

Qdrant (embeddings, 768d, cosine): intent-filtered top-5 + similarity

Drafter RAG: grounded reply, `sanitize_reply()` strips `http/www/t.co`

Escalation v5: legal | fraud | safety | PII | public-ID | repeat | human-ask | unclear

Output: AUTO-HANDLE vs ESCALATE + `triggered_by` + reason

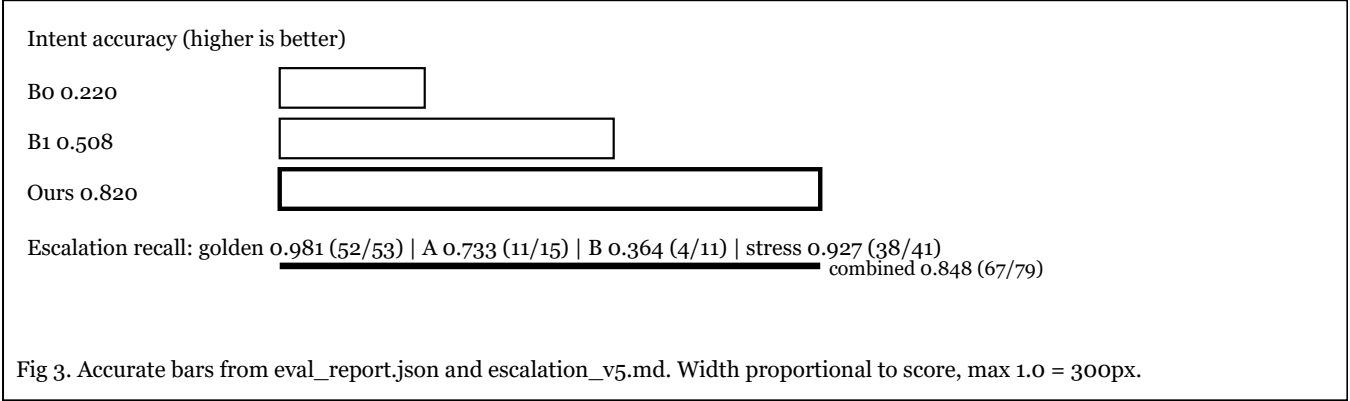
Fig 2. Accurate module map. `calibrate_confidence` is stored, gating still uses raw conf below 0.45 in shipped code.

The seven intents are `ORDER_STATUS`, `RETURN_REFUND`, `ACCOUNT_ACCESS`, `PRIME_SUBSCRIPTION`, `DEVICE_TECH_SUPPORT`, `DELIVERY_DAMAGE`, and `GENERAL_COMPLAINT` as catch-all. The noisiest boundaries are PRIME versus ACCOUNT for billing confusion and ORDER versus DAMAGE for late versus broken delivery. Banking77 was not used because banking intents do not transfer to retail shop support.

Unfiltered retrieval consistency at one is only 0.49, which justifies the intent filter, and RAG beats no-retrieval by 0.88 judge points on a fifty-row ablation. Experimental layers in `demo.py` including precedent agreement, grounding score, and calibrated floor are used only for the stress test, not for the shipped harness.

3. Results

Golden 250 results: B0 majority 0.220 accuracy, B1 TF-IDF fair 0.508 accuracy, our agent 0.820 accuracy and 0.809 macro-F1. Escalation on golden is recall 0.981 with 52 of 53 caught, false-auto rate 0.019, precision 0.867. The worst class is PRIME_SUBSCRIPTION with recall 0.429, where 6 cases go to ORDER_STATUS and 8 go to GENERAL_COMPLAINT. Per-class metrics and confusion matrix are in eval_report.json.

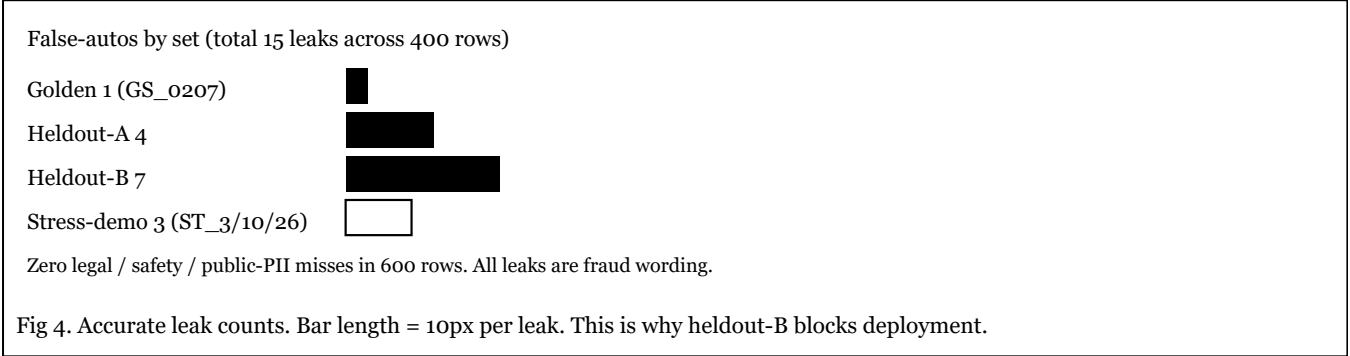


Frozen generalization is weaker. Heldout-A recall is 0.733 with 11 of 15 caught. Heldout-B recall is 0.364 with 4 of 11 caught. Combined frozen 350 cases give recall 0.848 with 67 of 79 caught and precision 0.838. Adversarial stress with demo policy gives recall 0.927 with 38 of 41 caught.

Data flow was twcs 2.8M rows to 155k threads to 10k corpus with 9,753 indexed after eval exclusion, to golden 200 plus 50 rare-class top-up for 250 total, plus cal 100 plus cal2 100 for tuning only and two frozen heldouts. Leakage checks pass with zero overlap and zero same-thread top-5. Human blind A/B ranks agent above template, but absolute judge grades are untrusted.

4. Failures

First, fraud paraphrase gap is the ship-blocker with 12 frozen false-autos plus 3 stress misses. Examples are GS_0207 with fladuent typo for fraudulent, H_0019 with charged twice, and HB_0047 with charged but not returned. Patterns memorized calibration wording and missed new phrasings. Second, PRIME versus ACCOUNT boundary collapse has pair accuracy 0.527. A prompt surgery attempt improved the pair by 0.018 but dropped overall accuracy from 0.80 to 0.76, so it was reverted.

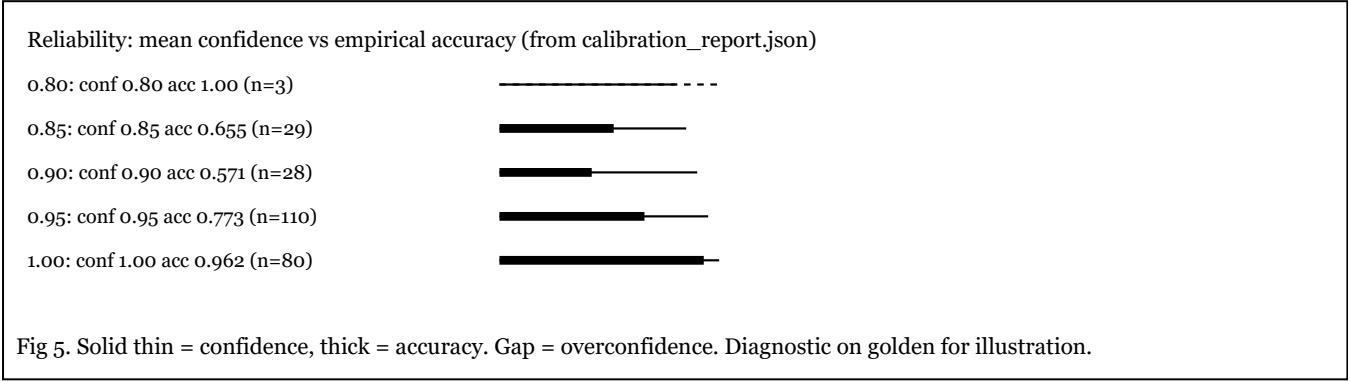


Third, the drafter once hallucinated t.co links that the judge missed, fixed by prompt rule plus regex strip to reach zero URLs in 250 replies. Fourth, keyword stratification did not equal label stratification, with only 8 DEVICE and 14 PRIME in the first 200, fixed by 50 outside-corpus human-labelled top-up rows to reach 24 and 28 at a cost of accuracy from 0.88 to 0.82.

Fifth, judge grades are untrustworthy with near-constant fives and safety inversions, mitigated by deterministic safety caps and demotion to direction only. Over-escalations include venting such as second time this month triggering repeat-contact, Japanese ultra-short messages routed to humans, and policy history references tripping loop detection. The headline 0.981 is golden-only, judge 24.44 runs four points high, and 52.4 percent auto-rate is a sweep optimum, not deployment coverage.

5. Method and safety

Calibration used only cal-100 plus cal2-100 with 200 rows, then locked v5 rules before measuring frozen sets once. Raw confidence is overconfident in the 0.80 to 1.00 band with mean 0.947 versus accuracy 0.80 and ECE 0.1518. The calibrate function maps 1.00 to 0.962, 0.95 to 0.773, and 0.90 to 0.571, stored in the response but not yet gating shipped routing, where the calibrated floor below 0.70 lives only in demo.py.

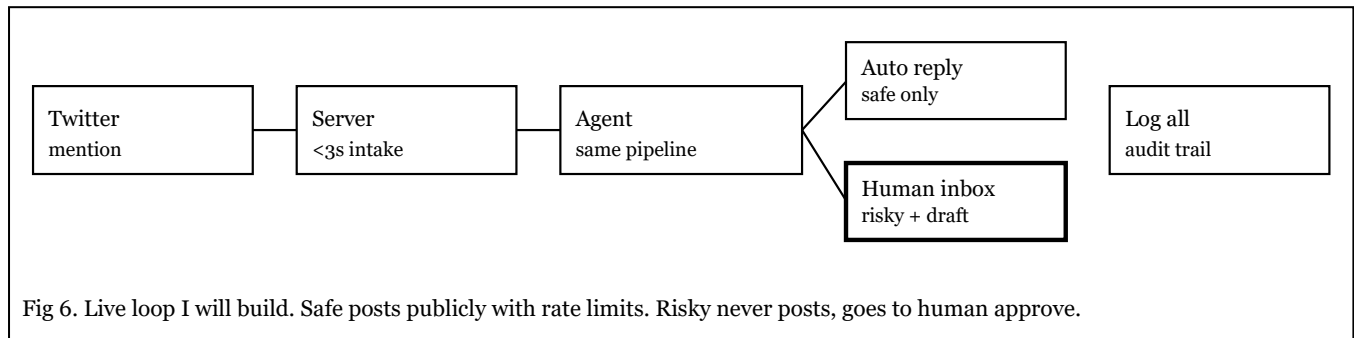


Trust comes from separation and disclosure. The old contaminated baseline was withdrawn, blind scoring used hidden source keys, zero legal and safety and public-PII misses occurred in 600 rows, and all misses are fraud wording gaps listed by ID. Tests include 27 offline pytest cases covering taxonomy, escalation contract, leakage hygiene, URL guardrail, judge safety rules, and error quarantine.

What was not built: multi-turn state, live Twitter connection, sentiment-shift tracking, fine-tuned embeddings, and PII redaction beyond the public order-ID escalation rule. The index manifest records 9,753 points with eval exclusion true. Those gaps are future work in the next page.

6. Next week: real-time Twitter chatbot

Next week I will build a real-time chatbot on Twitter and connect this agent there to solve live user queries. I will create a Twitter Developer app with elevated access, subscribe to mentions of the support handle with filtered stream, and run a small server that receives each new tweet within seconds. Each tweet will go through the same classify, retrieve, draft, and decide pipeline.



Safe cases will post a public reply automatically through the API with rate limits and duplicate guards. Risky cases will not post publicly and will go to a human inbox with reason and suggested draft for one-click approve or edit. I will add safety first with an allow-list and block-list, no links and no order IDs or personal data in public replies, opt-out handling, and a kill switch to stop auto-posting instantly.

Every decision will be logged with intent, confidence, similarity, and reason for audit. I will then pilot for seven days on about 200 live mentions with a daily cap, measuring auto-rate, false-autos by human review, median response time under one minute, and user satisfaction. Success is zero leaks with fast helpful replies. Architectural decisions are documented in DECISIONS.md across fifteen entries; core tools include TWCS dataset, foundation LLMs, Qdrant vector database, scikit-learn, and Streamlit.
